

Question Booklet

- The final exam contains 9 QUESTIONS and is scheduled for 4 HOURS.
- The maximum amount of points you can earn is 115.
- The minimum number of points you need to obtain in order to pass the exam is 55.
- Please verify if your question booklet consists of 12 PAGES (plus 4 pages with supplementary lecture slides) that are printed legibly, else contact your tutor immediately.
- The exam questions address both the theory and exercise parts of the lecture. For questions marked with an asterisk (*), supplementary lecture slides are provided at the end of this booklet.
- No electronic devices (such as calculators, notebooks, cell phones, PDAs) are allowed.
- Answers without sufficient details are void (e.g.: you can't just say "yes" or "no" as the answer).
- Only the answer booklet will be collected, please do not use the question booklet for answering the questions.

FROM ERM'S TO RELATIONAL DESIGNS

25 POINTS

Problem 1. Consider the following properties, relationships, and constraints describing the 7 entity sets *University*, *Course*, *Professor*, *Student*, *Literature*, *Book*, and *Script*.

- A *University* has a *name*, an *address*, and a number of *faculties*. It can be uniquely identified by its *name*, and it employs multiple *Professors*, offers multiple *Courses*, and it maintains the registrations of multiple *Students*.
- A *Course* has a unique *name*. It is taught at exactly one *University* and taught by exactly one *Professor*. It may refer to multiple *Literature* items, and it may have multiple *Students* who are registered.
- A *Professor* has a unique *name*, may teach multiple *Courses*, but works only for one *University*.
- A *Student* has a *name*, may attend multiple *Courses*, and may be enrolled into multiple *Universities*. For each *University*, at which the student is enrolled, there is a unique *matriculation number* that the student obtains from this *University*.
- A *Literature* item has a *name*, which is not necessarily unique across all *Courses* that are offered at the various *Universities*. Its *name* is however unique in combination with the *name* of the *Course* that it refers to.
- A *Book* is a special kind of *Literature*, which has the additional property *publisher*.
- A *Script* is a special kind of *Literature*, which has a number of *Students* and exactly one *Professor* as authors.
- Caution: A *Book* may be borrowed only by one *Student* at a time!

Please answer the following questions.

- a) (*) Provide an E/R diagram according to the Entity-Relationship-Model (ERM) which captures the above description. Make sure that your E/R diagram properly includes all the *keys* and *multiplicities* indicated in this description. Also point out for which of these relationships a *referential integrity constraint* makes sense in your E/R diagram.

10 POINTS

- b) (*) Translate the above description into a relation schema in SQL by providing CREATE TABLE statements for each of the above entity sets. Introduce a separate table for a relationship only if this is absolutely necessary to avoid redundancies. Clearly indicate the PRIMARY KEY and FOREIGN KEY dependencies that hold for your relation schema.

(Note: you may either use the E/R diagram you provided in a) or directly start from the above description to answer this question.)

10 POINTS

- c) (*) Determine in which of the Normal Forms (among those presented in the lecture) the relation schema you provided in b) is. You may assume that the only non-trivial functional dependencies that hold for your relations are those implied by the keys of your relation schema. Recall that a key uniquely determines all the remaining attributes of a relation. Please justify your answer.

5 POINTS

RELATIONAL DATA MODEL VS. SQL

12 POINTS

Problem 2. Briefly answer the following questions about the Relational Data Model and SQL.

- a) Characterize the properties of the Relational Data Model.

4 POINTS

- b) Give an example for a Natural Join operation over two relation instances of the schema $R(A, B)$ and $S(B, C)$ which yields a different result in Relational Algebra than in SQL. Provide your answer by showing the two relation instances for $R(A, B)$ and $S(B, C)$ and the two different results of the Natural Join (each for Relational Algebra and for SQL).

4 POINTS

- c) Can you find an SQL expression that is equivalent to the Natural Join in Relational Algebra for every possible instance of the two relations $R(A, B)$ and $S(B, C)$? Justify your answer by providing the corresponding SQL expression, or argue why this may not exist.

4 POINTS

RELATIONAL ALGEBRA VS. DATALOG

10 POINTS

Problem 3. Consider the two relations $R(A, B, C)$ and $S(A, B, C)$.

- a) Write the following expression formulated in Relational Algebra as a Datalog program.

$$\pi_{A,B}(R) \cup \rho_{B \rightarrow A, C \rightarrow B}(\pi_{B,C} \cdot \sigma_{A > 10 \text{ OR } B < 10}(S))$$

4 POINTS

- b) Determine whether the following Datalog program consists only of *safe* rules. If you think that this is the case, then provide a translation of the program into Relational Algebra; otherwise provide a justification for why the program may not be safe.

- $P(a, b) :- Q(a, b, c) \text{ AND NOT } R(a, b, c)$
- $Q(a, b, c) :- R(d, a, b) \text{ AND } S(a, b, c) \text{ AND NOT } a < b$

ABC A B C

6 POINTS

FUNCTIONAL DEPENDENCIES AND NORMAL FORMS

12 POINTS

Problem 4. Which of the following statements are true? Briefly justify your answer.

- (a) Every key of a relation schema is also a superkey of this relation schema.
2 POINTS
- (b) Every superkey of a relation schema is also a key of this relation schema.
2 POINTS
- (c) If there exist only trivial functional dependencies for a relation schema, then the only key of that relation schema is the set of all its attributes.
2 POINTS
- (d) If there exist only trivial multivalued dependencies for a relation schema, then the only key of that relation schema is the set of all its attributes.
2 POINTS
- (e) If the only key of a relation is the set of all its attributes, then the relation is BCNF.
2 POINTS
- (f) If the only key of a relation is the set of all its attributes, then the relation is 4NF.
2 POINTS

SQL QUERIES

12 POINTS

Problem 5. Consider the example data provided in the tables **Dealers**, **Producers**, **Products**, and **Deals** shown below.

dealer_id	name
1	Bob's Store
2	Appliances Inc.
3	Quality Goods
4	Tech Market

Table 1: Dealers

producer_id	name
1	United Technics
2	Technics Inc.
3	Big Bang
4	ACME

Table 2: Producers

product_id	type	price	producer_id	producer_id	dealer_id
1	vacuum cleaner	66.90	3	1	1
2	washing machine	399.99	2	1	2
3	tumble dryer	500.00	2	2	2
4	washing machine	780.00	3	2	4
5	oven	175.45	1	3	1
6	washing machine	545.90	1	3	2
7	vacuum cleaner	75.19	4	3	3
8	washing machine	519.99	4	4	2
9	tumble dryer	600.00	2	4	3

Table 3: Products

Table 4: Deals

You may assume that a dealer, who has a deal with a given producer, sells all the products produced by that producer.

- Give all types of products that are produced by at least two different producers.
3 POINTS
- Give the average price of a product sold by *Quality Goods*.
3 POINTS
- Give the name of the producer of the most expensive washing machine.
3 POINTS
- Give the names of the dealers that do not sell any vacuum cleaners.
3 POINTS

CONSTRAINTS AND TRIGGERS IN SQL

10 POINTS

Problem 6.

- a) Explain the difference between the evaluation of an ASSERTION and a (either value- or tuple-based) CHECK in SQL in order to enforce a constraint. Provide your answer by giving an example that shows this difference.

3 POINTS

- b) Given the following relational schema:

$R(A,B,C,D,E)$

Enforce the functional dependency $\{B,C\} \rightarrow \{D,E\}$ by writing an ASSERTION in SQL that expresses this constraint.

4 POINTS

- c) Assume you have two relations with the schemas *Products*(id, name, type, price, producer_id) and *Producers*(id, name, address), respectively.

The following view is defined over the two relations:

```
CREATE VIEW ProducersOfProducts AS
  SELECT Products.id as ProductID, Products.name as ProductName, type,
         Producers.id as ProducerID, Producers.name as ProducerName
  FROM Products, Producers
  WHERE producer_id = ProducerID;
```

Finally, the following trigger is defined:

```
CREATE TRIGGER NoInsertIntoView INSTEAD OF INSERT ON ProducersOfProducts
  REFERENCING NEW ROW AS NewRow FOR EACH ROW
  BEGIN
    INSERT INTO Products(id, name, type)
      VALUES (NewRow.ProductID, NewRow.ProductName, NewRow.type);
    INSERT INTO Producers(id, name)
      VALUES (NewRow.ProducerID, NewRow.ProducerName);
  END;
```

Explain in detail what the purpose of this trigger is and why this trigger is needed. Explain your answer by giving an example for the usage of the trigger.

3 POINTS

SCHEMA NORMALIZATION

12 POINTS

Problem 7. Consider the relation instance **Orders** as it is shown in Table 5. Suppose we know that the functional dependencies $client_id \rightarrow client_name$ and $\{shop_id, client_id, item_id\} \rightarrow delivery_date$ as well as the multivalued dependency $client_id \twoheadrightarrow client_address$ hold for this relation.

shop_id	client_id	item_id	price	delivery_date	client_address	client_name
1	1201	4	\$10	14/01/2013	17, Blank Avenue	Jane Doe
1	1201	27	\$10	20/01/2013	17, Blank Avenue	Jane Doe
1	2741	5	\$16	15/01/2013	28, High Road	Jack Russell
1	2741	5	\$16	15/01/2013	75, Low Boulevard	Jack Russell
2	1444	17	\$23	18/01/2013	206, E-Street	Jack Daniels
2	2861	14	\$11	13/02/2013	48, Blue Square	Bob Rope

Table 5: **Orders**

- Briefly explain the problems that may arise with this relation schema.
2 POINTS
- Determine the key(s) of this relation schema. Briefly justify your answer.
2 POINTS
- (*) Check whether this relation schema is in Boyce-Codd Normal Form (BCNF). Briefly justify your answer.
2 POINTS
- (*) Check whether this relation schema is in 4th Normal Form (4NF). Briefly justify your answer.
2 POINTS
- How would you normalize this relation schema, and what are the relation instances that you obtain for your normalized schema?
4 POINTS

STORAGE AND INDEXING

12 POINTS

Problem 8. Suppose we are given a relation schema $R(A, B)$, and we would like to organize the records that are stored under this relation schema in a sequential file with a fixed block size of $b = 2$. Further assume that we would like to create two index structures to support an efficient access to these records by the values of the A and B attributes, respectively. We intend to do this by creating

- a primary index on the A attribute, and
- a secondary index on the B attribute.

Please answer the following questions.

- Suppose we decide to store the records for $R(A, B)$ in a sequential file that is sorted according to the values of the A attribute. Does in this case a *sparse, 1st-level index* make sense to index the values stored under B ? Justify your answer.
2 POINTS
- Consider the instance of R that is depicted in Table 6. Index the values of the A attribute using a B^+ -tree index of order $n = 3$. You need to show only the final configuration of the B^+ -tree after all values have been inserted.
4 POINTS
- Consider again the instance of R that is depicted in Table 6. This time, index the values of the B attribute using either the *extensible* or the *linear hashing scheme* presented in the lecture. Show all the parameters used by the final configuration of your hashing scheme. (Note: to simplify the solution, the values of B are depicted already in binary format.)
4 POINTS
- Would you generally prefer the *extensible* or the *linear hashing scheme* presented in the lecture? Justify your answer.
2 POINTS

A	B
10	000111
5	010101
2	111111
1	111000
17	101100
5	100000
12	111101
8	111011
11	100011

Table 6: R

XML DATA MODEL

10 POINTS

Problem 9. Suppose we would like to compose a datastructure for a *directed graph* as an XML document.

- a) Consider the XML-Schema specification as it is depicted in Figure 1. Briefly describe the *key* and *foreign-key* constraints this XML-Schema file imposes over an XML document that is valid with respect to this schema.
2 POINTS
- b) Compare the XML document depicted in Figure 2 with the XML-Schema file of Figure 1 and determine whether the DTD provided in the preamble of the XML document indeed expresses the *same* key and foreign-key constraints. Justify your answer.
2 POINTS
- c) Is the XML document provided in Figure 2 *valid* with regard to the DTD and with regard to the XML-Schema file, respectively? Justify your answer.
2 POINTS
- d) (*) Provide an XQuery expression which, for every node, counts the number of its outgoing edges for the XML document provided in Figure 2. Your XQuery expression should return the value of each `nodeId` attribute together with this count.
Note: you may disregard both the DTD and the XML-Schema file to answer this question.
4 POINTS


```

<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="DIRECTED_GRAPH">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="NODE" minOccurs="0" maxOccurs="unbounded">
          <xs:complexType>
            <xs:attribute name="nodeId" type="xs:string" use="required" />
            <xs:attribute name="nodeLabel" type="xs:string" use="optional" />
          </xs:complexType>
        </xs:element>
        <xs:element name="EDGE" minOccurs="0" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="SOURCE_NODE" minOccurs="1" maxOccurs="1">
                <xs:complexType>
                  <xs:attribute name="sourceId" type="xs:string" use="required" />
                </xs:complexType>
              </xs:element>
              <xs:element name="TARGET_NODE" minOccurs="1" maxOccurs="1">
                <xs:complexType>
                  <xs:attribute name="targetId" type="xs:string" use="required" />
                </xs:complexType>
              </xs:element>
            </xs:sequence>
            <xs:attribute name="edgeId" type="xs:string" use="required" />
            <xs:attribute name="edgeLabel" type="xs:string" use="optional" />
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="graphName" type="xs:string" use="optional" />
    </xs:complexType>
    <xs:key name="nodeKey">
      <xs:selector xpath="./NODE" />
      <xs:field xpath="@nodeId" />
    </xs:key>
    <xs:key name="edgeKey">
      <xs:selector xpath="./EDGE" />
      <xs:field xpath="@edgeId" />
    </xs:key>
    <xs:keyref name="sourceNodeRef" refer="nodeKey">
      <xs:selector xpath="./EDGE/SOURCE_NODE" />
      <xs:field xpath="@sourceId" />
    </xs:keyref>
    <xs:keyref name="targetNodeRef" refer="nodeKey">
      <xs:selector xpath="./EDGE/TARGET_NODE" />
      <xs:field xpath="@targetId" />
    </xs:keyref>
  </xs:element>
</xs:schema>

```

Figure 1: An XML-Schema file describing a *directed graph* datastructure.


```

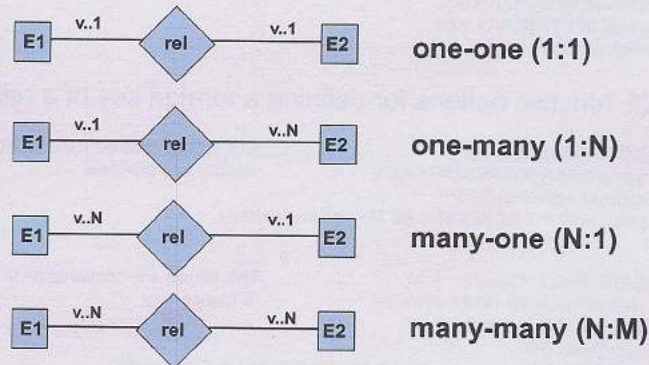
<?xml version="1.0" standalone="no"?>
<!DOCTYPE DIRECTED_GRAPH [
  <!ELEMENT DIRECTED_GRAPH (NODE*, EDGE*)>
  <!-- ATTLIST DIRECTED_GRAPH graphName CDATA #IMPLIED -->
  <!ELEMENT NODE EMPTY>
  <!-- ATTLIST NODE nodeId ID #REQUIRED -->
  <!-- ATTLIST NODE nodeLabel CDATA #IMPLIED -->
  <!ELEMENT EDGE (SOURCE_NODE, TARGET_NODE)>
  <!-- ATTLIST EDGE edgeId ID #REQUIRED -->
  <!-- ATTLIST EDGE edgeLabel CDATA #IMPLIED -->
  <!ELEMENT SOURCE_NODE EMPTY>
  <!-- ATTLIST SOURCE_NODE sourceId IDREF #REQUIRED -->
  <!ELEMENT TARGET_NODE EMPTY>
  <!-- ATTLIST TARGET_NODE targetId IDREF #REQUIRED -->
]>
<DIRECTED_GRAPH graphName="MyGraph-2.0">
  <NODE nodeId="A" nodeLabel="Node A" />
  <NODE nodeId="B" nodeLabel="Node B" />
  <NODE nodeId="C" nodeLabel="Node C" />
  <EDGE edgeId="A" edgeLabel="Edge A=>B">
    <SOURCE_NODE sourceId="A" />
    <TARGET_NODE targetId="B" />
  </EDGE>
  <EDGE edgeId="B" edgeLabel="Edge B=>C">
    <SOURCE_NODE sourceId="B" />
    <TARGET_NODE targetId="C" />
  </EDGE>
  <EDGE edgeId="C" edgeLabel="Edge A=>C">
    <SOURCE_NODE sourceId="A" />
    <TARGET_NODE targetId="C" />
  </EDGE>
</DIRECTED_GRAPH>

```

Figure 2: An XML document capturing a DTD and an instance of a *directed graph*.

Multiplicities of Relationships in ERM

$$v \in \{0,1\}$$



Summary of Translations

- **Classes/interfaces** in ODL and **entity sets** in ERM are each translated by creating a new relation.
 - All **attributes** (incl. structs and collection types in ODL) are taken over.
 - **Multi-valued attributes** are flattened by creating a new tuple for each combination of values stored in the multi-valued attributes.
 - **Weak entity sets** and **subclasses** take over the key attributes of their key-supplying and super-class(es), respectively; key and foreign-key references are added.
- **Higher-arity relationships** in ERMs are first translated into binary relationships. (→ Exercise Sheet #1)
- **1:1 relationships** are each translated by adding foreign-key references to both the source and the target relation.
- **1:N relationships** are each translated by adding a foreign-key reference to the target relation. (and vice-versa)
- **N:M relationships** are each translated by creating a new relation, which takes over the key attributes of the source and target relation, and by adding foreign-key references to both the source and the target relation.



Defining Foreign Keys

```
CREATE TABLE MovieExec (  
  name CHAR(30) UNIQUE,  
  address VARCHAR(255),  
  certN INT PRIMARY KEY,  
  netWorth INT);
```

SQL has two options for defining a foreign key of a relation:

```
CREATE TABLE Studio (  
  name CHAR(30) PRIMARY KEY,  
  address VARCHAR(255),  
  presCertN INT REFERENCES MovieExec(certN));
```

This is only possible if the foreign key contains exactly one attribute.

```
CREATE TABLE Studio (  
  name CHAR(30) PRIMARY KEY,  
  address VARCHAR(255),  
  presCertN INT,  
  FOREIGN KEY (presCertN) REFERENCES MovieExec(certN));
```

This allows any combination of attributes to be a foreign key.

A Complete Set of Inference Rules

Armstrong's Axioms:

The following three axioms allow for deriving any FD that follow from a given set of FD's.

1. Reflexivity:

If $\{B_1, B_2, \dots, B_m\} \subseteq \{A_1, A_2, \dots, A_n\}$,
then $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$. ($\rightarrow$ "trivial FDs")

2. Augmentation:

If $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$,
then $A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k \rightarrow B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k$,
for any set of attributes $C_1, C_2, \dots, C_k$. ($\rightarrow$ "non-trivial FDs")

3. Transitivity:

If $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ and $B_1, B_2, \dots, B_m \rightarrow C_1, C_2, \dots, C_k$,
then $A_1, A_2, \dots, A_n \rightarrow C_1, C_2, \dots, C_k$. ($\rightarrow$ "closure algorithm")

Normal Forms of Relations

In the following, let X be a *set of attributes* and Y be a *single attribute* of a relation R , and let F be a given set of FD's on R .

R is in **First Normal Form** (1NF), if all attribute values of every tuple are atomic. All the relations we consider are in 1NF.

R is in **Second Normal Form** (2NF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is not a proper subset of a key of R or Y is an attribute of a key of R .

R is in **Third Normal Form** (3NF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R or Y is an attribute of a key of R .

R is in **Boyce-Codd Normal Form** (BCNF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R .



Fourth Normal Form

A MVD $A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$ over a relation R is called **non-trivial** if

- none of the B 's is among the A 's;
- not all of the attributes of R are among the A 's and B 's.

A relation is in **Fourth Normal Form** (4NF), if for every non-trivial MVD $A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$ over R it holds that $\{A_1, A_2, \dots, A_n\}$ is a superkey of R .

→ Note that the definition of “keys” and “superkeys” still depends on FD's only!



XQuery vs. Relational Algebra (IV)

- Special case: suppose we want to just filter a sequence of nodes by another node sequence. Then one **for** loop and one **let** is enough.

Notice the "join" among nodes in the **\$x** and **\$y** sequences.

- Example:

```
for $x in doc("bars.xml")/BARS/BAR
let $y := doc("bars.xml")/BARS/BAR[./@name =
    $x/@name)]//PRICE
return <BAR><NAME>{$x/@name}</NAME></BAR>
```



XQuery vs. Relational Algebra (V)

- The common *aggregations* **count**, **sum**, **avg**, **min**, **max** can be applied (as functions) to sequences in XQuery as well.

- Example:

```
let $bar := doc("bars.xml")//BAR
return <COUNT>{count($bar)}</COUNT>
```

